Data Science Lab

Name: Vishal Rajesh Mahajan Experiment No: 8

Class: BE INFT B-1 Roll No: 1

To implement Supervised Learning algorithm - Random Forest

Import Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import kagglehub
from kagglehub import KaggleDatasetAdapter
```

Load the dataset

```
df = kagglehub.load_dataset(
  KaggleDatasetAdapter.PANDAS,
  "pavansubhasht/ibm-hr-analytics-attrition-dataset","WA_Fn-UseC_-HR-Employee-Attrition.csv"
)
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (1470, 35)
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	...	RelationshipSatisfaction
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1	...	1
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2	...	4
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	4	...	2
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	5	...	3
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	7	...	4

5 rows × 35 columns

Drop constant / identifier columns

```
df = df.drop(['Over18','EmployeeCount', 'EmployeeNumber', 'StandardHours'], axis=1)
```

Encode categorical variables

```
le = LabelEncoder()
```

Encode target first

```
df['Attrition'] = le.fit_transform(df['Attrition'])
df['Attrition']
```

	Attrition
0	1
1	0
2	1
3	0
4	0
...	...
1465	0

Encode remaining categorical columns

```
categorical_cols = ['BusinessTravel', 'Department', 'EducationField', 'Gender',  'JobRole',
'MaritalStatus', 'OverTime']
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```

Split features and target

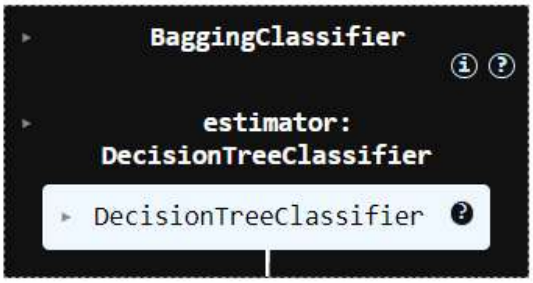
```
X = df.drop('Attrition', axis=1)
y = df['Attrition']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Define base estimator and BaggingClassifier

```
base_tree = DecisionTreeClassifier(max_depth=8, random_state=42)
bag_model = BaggingClassifier(
    estimator=base_tree,
    n_estimators=50,
    random_state=42,
    bootstrap=True
)
```

Train Bagging model

```
bag_model.fit(X_train, y_train)
```



Evaluate model

```
y_pred = bag_model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.8469387755102041

Classification Report:
              precision    recall  f1-score   support

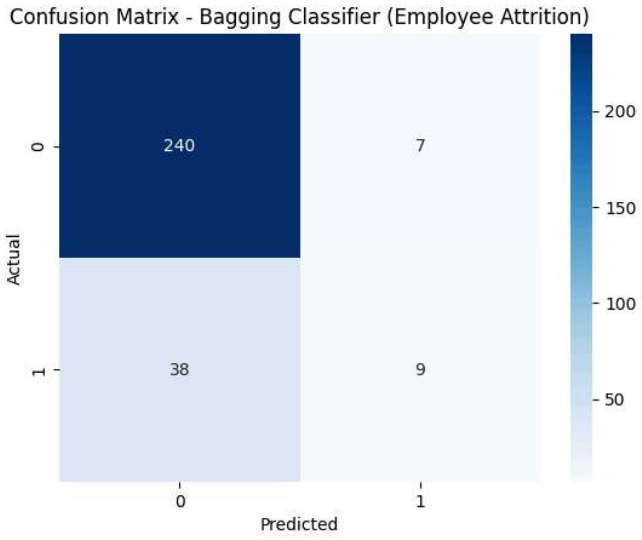
           0       0.86      0.97      0.91       247
           1       0.56      0.19      0.29        47

    accuracy                           0.85       294
   macro avg       0.71      0.58      0.60       294
weighted avg       0.82      0.85      0.81       294
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Bagging Classifier (Employee Attrition)")
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
plt.show()
```



Inference on a new employee

```
sample_employee = {
    'Age': 35,
    'BusinessTravel': 1,
    'DailyRate': 1100,
    'Department': 2,
    'DistanceFromHome': 10,
    'Education': 3,
    'EducationField': 2,
    'EnvironmentSatisfaction': 2,
    'Gender': 1,
    'HourlyRate': 65,
    'JobInvolvement': 3,
    'JobLevel': 2,
    'JobRole': 4,
    'JobSatisfaction': 2,
    'MaritalStatus': 0,
    'MonthlyIncome': 6500,
    'MonthlyRate': 20000,
    'NumCompaniesWorked': 2,
    'OverTime': 1,
    'PercentSalaryHike': 15,
    'PerformanceRating': 3,
```

```
    'RelationshipSatisfaction': 2,
    'StockOptionLevel': 0,
    'TotalWorkingYears': 10,
    'TrainingTimesLastYear': 3,
    'WorkLifeBalance': 3,
    'YearsAtCompany': 5,
    'YearsInCurrentRole': 3,
    'YearsSinceLastPromotion': 1,
    'YearsWithCurrManager': 3
}


sample_df = pd.DataFrame([sample_employee])
prediction = bag_model.predict(sample_df)
prediction_proba = bag_model.predict_proba(sample_df)

if prediction[0] == 1:
    print("\nPrediction: Employee is likely to LEAVE.")
else:
    print("\nPrediction: Employee is likely to STAY.")

print("Prediction Probabilities (Stay / Leave):", prediction_proba[0])
```



```
Prediction: Employee is likely to STAY.
Prediction Probabilities (Stay / Leave): [0.83522097 0.16477903]
```